



全站搜索

帮助

声明：百科词条人人可编辑，词条创建和修改均免费，绝不存在官方及代理商付费代编，请勿上当受骗。 [详情>>](#)



首页

分类

特色百科

用户

权威合作

手机百科

个人中心

BMP是一个**多义词**，请在下列**义项**上选择浏览（共**8**个义项）

[+ 添加义项](#)

▪ **图像文件格式(Bitmap)**

▪ **数字压力传感器**

▪ **BMP诚信借款(Business Person Media)**

▪ **生物甲烷势**

▪ **商业模式创新与资本运营**

▪ **骨形态发生蛋白**

▪ **基本多文种平面**

▪ **俄罗斯BMP系列步兵战车**

BMP（图像文件格式(Bitmap)）

[编辑](#)

收藏

945

83

BMP（全称**Bitmap**）是Windows操作系统中的标准图像文件格式，可以分成两类：设备相关**位图**（DDB）和设备无关位图（DIB），使用非常广。它采用位映射存储格式，除了图像深度可选以外，不采用其他任何压缩，因此，**BMP**文件所占用的空间很大。**BMP**文件的图像深度可选**1bit**、**4bit**、**8bit**及**24bit**。**BMP**文件存储数据时，图像的扫描方式是按从左到右、从下到上的顺序。由于**BMP**文件格式是Windows环境中交换与图有关的数据的一种标准，因此在Windows环境中运行的图形图像软件都支持**BMP**图像格式。

中文名	图像文件格式	简 称	BMP
外文名	Bitmap	系 统	
		Windows操作系统	应 用
		设备相关 位图 设备无关位图	

目录

- 格式组成
- 格式类型
- 对应数据结构
- 读取方法
- 文件部分



BMP图册

V百科

往期回顾



相关文件格式

纠错



[jpg](#)



[tiff](#)



[png](#)

分享



秒懂星课堂
**张馨予：为了TA
我变身花木兰**



格式组成

典型的BMP图像文件由四部分组成：

- 位图头文件数据结构，它包含**BMP**图像文件的类型、显示内容等信息；
- 位图**信息数据结构，它包含有**BMP**图像的宽、高、压缩方法，以及定义颜色等信息；
- 调色板**，这个部分是可选的，有些位图需要调色板，有些位图，比如真彩色图（**24**位的**BMP**）就不需要调色板；
- 位图数据，这部分的内容根据**BMP**位图使用的位数不同而不同，在**24**位图中直接使用**RGB**，而其他的小于**24**位的使用调色板中颜色索引值。

格式类型

位图一共有两种类型，即：设备相关位图（**DDB**）和设备无关位图（**DIB**）。**DDB**位图在早期的**Windows**系统（**Windows 3.0**以前）中是很普遍的，事实上它也是唯一的。然而，随着显示器制造技术的进步，以及显示设备的多样化，**DDB**位图的一些固有的问题开始浮现出来了。比如，它不能够存储（或者说获取）创建这张图片的原始设备的分辨率，这样，应用程序就不能快速的判断客户机的显示设备是否适合显示这张图片。为了解决这一难题，微软创建了**DIB****位图**格式。

设备无关位图 (Device-Independent Bitmap)

DIB位图包含下列的颜色和尺寸信息：

- * 原始设备（即创建图片的设备）的颜色格式。
- * 原始设备的分辨率。
- * 原始设备的调色板
- * 一个**位数****组**，由红、绿、蓝（**RGB**）三个值代表一个像素。
- * 一个数组压缩标志，用于表明数据的压缩方案（如果需要的话）。

以上这些信息保存在**BITMAPINFO**结构中，该结构由**BITMAPINFOHEADER**结构和两个或更多个**RGBQUAD**结构所组成。**BITMAPINFOHEADER**结构所包含的成员表明了图像的尺寸、原始设备的颜色格式、以及数据压缩方案等信息。**RGBQUAD**结构标识了像素所用到的颜色数据。

DIB位图也有两种形式，即：底到上型**DIB(bottom-up)**，和顶到下型**DIB(top-down)**）。底到上型**DIB**的**原点（origin）**在图像的左下角，而顶到下型**DIB**的原点在图像的左上角。如果**DIB**的高度值（由**BITMAPINFOHEADER**结构中的**biHeight**成员标识）是一个正值，那么就表明这个**DIB**是一个底到上型**DIB**，如果高度值是一个负值，那么它就是一个顶到下型**DIB**。注意：顶到下型的**DIB**位图是不能被压缩的。

位图的颜色格式是通过颜色面板值（**planes**）和颜色位值（**bitcount**）计算得来的，颜色面板值永远是**1**，而颜色位值则可以是**1**、**4**、**8**、**16**、**24**、**32**其中的一个。如果它是**1**，则表示位图是一张单色位图（译者注：通常是黑白位图，只有黑和白两种颜色，当然它也可以是任意两种指定的颜色），如果它是**4**，则表示这是一张**VGA**位图，如果它是**8**、**16**、**24**、或是**32**，则表示该位图是其他设备所产生的位图。如果应用程序想获取当前显示设备（或打印机）的颜色位值（或称位深度），可调用**API**函数**GetDeviceCaps**（），并将第二个参数设为**BITSPIXEL**即可。

编辑

		
 dxf 	 wav 	 xls
		
 eps格式 	 png图标 	 jpg图片
 相关软件	 纠错	
		
 视频去马... 	 狸窝全能... 	 fraps
		
 美图看看 	 图片浏览器 	 格式工厂
 相关术语	 纠错	
		
 bpm 	 二值图像 	 位图



词条统计

浏览次数：1032532次

编辑次数：98次**历史版本**

最近更新：2017-07-19

创建者：**shdiao**

- | | | | |
|----------|---------|-----------|----------|
| 1 | 全息投影多少钱 | 12 | ui设计师薪水 |
| 2 | 图像尺寸测量仪 | 13 | 图像无线传输 |
| 3 | 55寸拼接屏 | 14 | d1p无拼接大屏 |
| 4 | 大数据分析培训 | 15 | 拼接屏价格 |
| 5 | 图像检测技术 | 16 | 图像算法工程师 |
| 6 | 视觉图像检测 | 17 | 车牌识别 |
| 7 | 大恒图像 | 18 | 图像ps课程 |

显示设备的分辨率是以每米多少个像素来表明的，应用程序可以通过以下三个步骤来获取显示设备或打印机的[水平分辨率](#)：

1. 调用GetDeviceCaps（）函数，指定第二个参数为HORZRES。
2. 再次调用GetDeviceCaps()函数，指定第二个参数为HORZSIZE。
3. 用第一个返回值除以第二个返回值。即：GetDeviceCaps(hDC,HORZRES)/GetDeviceCaps(hDC,HORZSIZE);

应用程序也可以使用相同的三个步骤来获取设备的垂直分辨率，不同之处只是要将HORZRES替换为VERTRES，把HORZSIZE替换为VERTSIZE，即可。

调色板是被保存在一个RGBQUAD结构的[数组](#)中，该结构指出了每一种颜色的红、绿、蓝的分量值。位数组中的每一个索引都对应于一个调色板项（即一个RGBQUAD结构），应用程序将根据这种对应关系，将像素索引值转换为像素RGB值（真实的像素颜色）。应用程序也可以通过调用GetDeviceCaps（）函数来获取当前显示设备的调色板尺寸（将该函数的第二个参数设为NUMCOLORS即可）。

Win32 API支持位数据的压缩（只对8位和4位的底到上型DIB位图）。压缩方法是采用运行长度编码方案（RLE），RLE使用两个字节来描述一个句法，第一个字节表示重复像素的个数，第二个字节表示重复像素的索引值。有关压缩位图的详细信息请参见对BITMAPINFOHEADER结构的解释。

应用程序可以从一个DDB位图创建出一个DIB位图，步骤是，先初始化一些必要的结构，然后再调用GetDIBits（）函数。不过，有些显示设备有可能不支持这个函数，你可以通过调用GetDeviceCaps（）函数来确定一下（GetDeviceCaps（）函数在调用时指定RC_DI_BITMAP作为RASTERCAPS的标志）。

应用程序可以用DIB去设置显示设备上的像素（译者注：也就是显示DIB），方法是调用SetDIBitsToDevice（）函数或调用StretchDIBits（）函数。同样，有些显示设备也有可能不支持以上这两个函数，这时你可以指定RC_DIBTODEV作为RASTERCAPS标志，然后调用GetDeviceCaps（）函数来判断该设备是否支持SetDIBitsToDevice（）函数。也可以指定RC_STRETCHDIB作为RASTERCAPS标志来调用GetDeviceCaps（）函数，来判断该设备是否支持StretchDIBits（）函数。

如果应用程序只是要简单的显示一个已经存在的DIB位图，那么它只要调用SetDIBitsToDevice（）函数就可以。比如一个电子表格软件，它可以打开一个图表文件，在窗口中简单的调用SetDIBitsToDevice（）函数，将图形显示在窗口中。但如果应用程序要重复的绘制位图的话，则应该使用BitBlt（）函数，因为BitBlt（）函数的执行速度要比SetDIBitsToDevice()函数快很多。

设备相关位图 (Device-Dependent Bitmaps)

设备相关位图（DDB）之所以现在还被系统支持，只是为了兼容旧的Windows 3.0软件，如果程序员现在要开发一个与位图有关的程序，则应该尽量使用或生成DIB格式的位图。

DDB位图是被一个单个结构BITMAP所描述，这个结构的成员标明了该位图的宽度、高度、设备的颜色格式等信息。

DDB位图也有两种类型，即：可废弃的（discardable）DDB和不可废弃的（nondiscardable）DDB。可废弃的DDB位图就是一种当[系统内存](#)缺乏，并且该位图也没有被选入设备描述表（DC）的时候，系统就会把该DDB位图从内存中清除（即废弃）。不可废弃的DDB则是无论系统内存多少都不会被系统清除的DDB。API函数CreateDiscardableBitmap（）函数可用于创建可废弃位图。而函数CreateBitmap（）、CreateCompatibleBitmap（）、和CreateBitmapIndirect（）可用于创建不可废弃的位图。

应用程序可以通过一个DIB位图而创建一个DDB位图，只要先初始化一些必要的结构，然后再调用CreateDIBitmap（）函数就可以。如果在调用该函数时指定了CBM_INIT标志，那么这一次调用就等价于先调用CreateCompatibleBitmap（）创建当前设备格式的DDB位图，然后又调用SetDIBits（）函数转换DIB格式到DDB格式。（可能有些设备并不支持SetDIBits()函数，你可以指定RC_DI_BITMAP作为RASTERCAPS的标志，然后调用GetDeviceCaps（）函数来判断一下）。

8	观音图像	19	变形缝
9	图像物体识别	20	贷款图像
10	图像粒度仪	21	车牌识别系统
11	投影手机	22	英语记忆方法

广告

对应数据结构

[编辑](#)

1：BMP文件组成

BMP文件由[文件头](#)、位图信息头、颜色信息和[图形数据](#)四部分组成。

2：BMP文件头（14字节）

BMP文件头数据结构含有BMP文件的类型、文件大小和位图起始位置等信息。

其结构定义如下：

```
1 typedef struct tagBITMAPFILEHEADER
2 {
3     WORD  bfType; // 位图文件的类型，必须为BM(1-2字节)
4     DWORD bfSize; // 位图文件的大小，以字节为单位（3-6字节，低位在前）
5     WORD  bfReserved1; // 位图文件保留字，必须为0(7-8字节)
6     WORD  bfReserved2; // 位图文件保留字，必须为0(9-10字节)
7     DWORD bOfffBits; // 位图数据的起始位置，以相对于位图（11-14字节，低位在前）
8     // 文件头的偏移量表示，以字节为单位
9 } __attribute__((packed)) BITMAPFILEHEADER;
```

3：位图信息头（40字节）

BMP位图信息头数据用于说明位图的尺寸等信息。

```
1 typedef struct tagBITMAPINFOHEADER{
2     DWORD biSize; // 本结构所占字节数（15-18字节）
3     LONG  biWidth; // 位图的宽度，以像素为单位（19-22字节）
4     LONG  biHeight; // 位图的高度，以像素为单位（23-26字节）
5     WORD  biPlanes; // 目标设备的级别，必须为1(27-28字节)
6     WORD  biBitCount; // 每个像素所需的位数，必须是1（双色），（29-30字节）
7     // 4(16色），8(256色) 16(高彩色)或24（真彩色）之一
8     DWORD biCompression; // 位图压缩类型，必须是0（不压缩），（31-34字节）
9     // 1(BI_RLE8压缩类型)或2(BI_RLE4压缩类型)之一
10    DWORD biSizeImage; // 位图的大小(其中包含了为了补齐行数是4的倍数而添加的空字节)，以字节为单位（35-38字节）
11    LONG  biXPelsPerMeter; // 位图水平分辨率，每米像素数（39-42字节）
12    LONG  biYPelsPerMeter; // 位图垂直分辨率，每米像素数（43-46字节）
13    DWORD biClrUsed; // 位图实际使用的颜色表中的颜色数（47-50字节）
14    DWORD biClrImportant; // 位图显示过程中重要的颜色数（51-54字节）
15 } __attribute__((packed)) BITMAPINFOHEADER;
```

4：颜色表

颜色表用于说明位图中的颜色，它有若干个表项，每一个表项是一个RGBQUAD类型的结构，定义一种颜色。RGBQUAD结构的定义如下：

```
1 typedef struct tagRGBQUAD{
2     BYTE  rgbBlue; // 蓝色的亮度（值范围为0-255）
3     BYTE  rgbGreen; // 绿色的亮度（值范围为0-255）
4     BYTE  rgbRed; // 红色的亮度（值范围为0-255）
5     BYTE  rgbReserved; // 保留，必须为0
6 } __attribute__((packed)) RGBQUAD;
```

颜色表中RGBQUAD结构数据的个数有biBitCount来确定：

当biBitCount=1,4,8时，分别有2,16,256个表项；

当biBitCount=24时，没有颜色表项。

位图信息头和颜色表组成位图信息，BITMAPINFO结构定义如下：

```
1 typedef struct tagBITMAPINFO{
2     BITMAPINFOHEADER bmiHeader; // 位图信息头
3     RGBQUAD bmiColors[1]; // 颜色表
4 } __attribute__((packed)) BITMAPINFO;
```

5：位图数据

位图[数据记录](#)了位图的每一个像素值，记录顺序是在扫描行内是从左到右，扫描行之间是从下到上。位图的一个像素值所占的字节数：

当biBitCount=1时，8个像素占1个字节；

当biBitCount=4时，2个像素占1个字节；

当biBitCount=8时，1个像素占1个字节；

当biBitCount=24时，1个像素占3个字节,按顺序分别为B,G,R；

Windows规定一个扫描行所占的字节数必须是

4的倍数（即以long为单位），不足的以0填充，

biSizeImage = (((bi.biWidth * bi.biBitCount) + 31) & ~31) / 8) * bi.biHeight;

具体数据举例：

如某BMP文件开头：

4D42 46900000 0000 0000 4600 0000 2800 0000 8000 0000 9000 0000 0100*1000 0300 0000 0090 0000 A00F 0000
A00F0000 0000 00000000 0000*00F8 E007 1F00 0000*02F1 84F1 04F1 84F1 84F1 06F2 84F1 06F2 04F2 86F2 06F2 86F2
86F2

读取方法

[编辑](#)

/*

功能： 在图片的第50行画一条黑线

为简化代码，只支持24位色的图片

codeblocks下正确运行。VC下需要将二维数组img改为malloc动态分配。需要添加#include "stdlib.h"。

*/

```
1  #include<stdio.h>
2  #include<windows.h>
3  typedef struct
4  {
5      BYTE b;
6      BYTE g;
7      BYTE r;
8  }RGB;
9  int main(void)
10 {
11     BITMAPFILEHEADER fileHeader;
12     BITMAPINFOHEADER infoHeader;
13     FILE *pfin=fopen("原始图像.bmp","rb");
14     FILE *pfout=fopen("修改后的图像.bmp","wb");
15     //ReadtheBitmapfileheader;
16     fread(&fileHeader,sizeof(BITMAPFILEHEADER),1,pfin);
17     //ReadtheBitmapinfoheader;
18     fread(&infoHeader,sizeof(BITMAPINFOHEADER),1,pfin);
19     //为简化代码，只处理24位彩色
20     if(infoHeader.biBitCount==24)
21     {
22         int size=infoHeader.biWidth*infoHeader.biHeight;
23         RGB img[infoHeader.biHeight][infoHeader.biWidth]; //这里有错误，尺度改为常量
24         fread(img,sizeof(RGB),size,pfin);
25         //把第50行染成黑色
```

```
26     int i=0;
27     for(;i<infoHeader.biWidth;i++)
28     {
29         img[50][i].b=img[50][i].g=img[50][i].r=0;
30     }
31     //将修改后的图片保存到文件
32     fwrite(&fileHeader,sizeof(fileHeader),1,pfout);
33     fwrite(&infoHeader,sizeof(infoHeader),1,pfout);
34     fwrite(img,sizeof( RGB),size,pfout);
35 }
36 fclose(pfin);
37 fclose(pfout);
38 }
```

文件部分

[□ 编辑](#)

图像文件头

1) 1-2: (这里的数字代表的是字节，下同) 图像文件头。0x4d42='BM'，表示是Windows支持的BMP格式。(注意：查ascii表B 0x42,M0x4d,bfType 为两个字节，B为low字节，M为high字节所以bfType=0x4D42，而不是0x424D，请注意)

2) 3-6: 整个文件大小。4690 0000，为00009046h=36934。

3) 7-8: 保留，必须设置为0。

4) 9-10: 保留，必须设置为0。

5) 11-14: 从文件开始到位图数据之间的偏移量(14+40+4*（2^biBitCount）)(在有颜色板的情况下)。4600 0000，为00000046h=70，上面的文件头就是35字=70字节。

位图信息头

6) 15-18: 位图图信息头长度。

7) 19-22: 位图宽度，以像素为单位。8000 0000，为00000080h=128。

8) 23-26: 位图高度，以像素为单位。9000 0000，为00000090h=144。

9) 27-28: 位图的位面数，该值总是1。0100，为0001h=1。

10) 29-30: 每个像素的位数。有1（单色），4（16色），8（256色），16（64K色，高彩色），24（16M色，真彩色），32（4096M色，增强型真彩色）。1000为0010h=16。

11) 31-34: 压缩说明：有0（不压缩），1（RLE 8，8位RLE压缩），2（RLE 4，4位RLE压缩，3（Bitfields，位域存放）。RLE简单地说是采用像素数+像素值的方式进行压缩。T408采用的是位域存放方式，用两个字节表示一个像素，位域分配为r5b6g5。图中0300 0000为00000003h=3(这张图片不存在颜色板)。

12) 35-38: 用字节数表示的位图数据的大小，该数必须是4的倍数，数值上等于:一行所占的字节数×位图高度。0090 0000为00009000h=80×90×2h=36864。假设位图是24位,宽为41，高为30，则数值=(biWidth*biBitCount+31)/32*4*biHeight,即=(41*24+31)/32*4*30=3720

13) 39-42: 用像素/米表示的水平分辨率。A00F 0000为0000 0FA0h=4000。

14) 43-46: 用像素/米表示的垂直分辨率。A00F 0000为0000 0FA0h=4000。

15) 47-50: 位图使用的颜色索引数。设为0的话，则说明使用所有调色板项。

16）51-54：对图象显示有重要影响的颜色索引的数目。如果是0，表示都重要。

彩色板

17）（55+0）到（50-1+2^biBitCount）：彩色板规范。对于调色板中的每个表项，用下述方法来描述RGB的值：

1字节用于蓝色分量

1字节用于绿色分量

1字节用于红色分量

1字节用于填充符（设置为0）

对于24-位真彩色图像就不使用彩色板，因为位图中的RGB值就代表了每个象素的颜色。

如，彩色板为00F8 0000 E007 0000 1F00 0000 0000 0000，其中：

00F8为F800h = 1111 1000 0000 0000（二进制），是蓝色分量的掩码。

E007 为 07E0h = 0000 0111 1110 0000（二进制），是绿色分量的掩码。

1F00为001Fh = 0000 0000 0001 ^[1] 1111（二进制），是红色分量的掩码。

0000 总设置为0。

将掩码跟像素值进行“与”运算再进行移位操作就可以得到各色分量值。看看掩码，就可以明白事实上在每个像素值的两个字节16位中，按从高到低取5、6、5位分别就是r、g、b分量值。取出分量值后把r、g、b值分别乘以8、4、8就可以补齐第个分量为一个字节，再把这三个字节按rgb组合，放入存储器（同样要反序），就可以转换为24位标准BMP格式了。

图像数据阵列

18)55（无调色板）-bfSize：每两个字节表示一个像素。阵列中的第一个字节表示位图左下角的像素，而最后一个字节表示位图右上角的像素。

//----图像处理-----BMP为DIB类型，从底向上显示-----

//阵列中的第一个字节表示位图左下角的像素，而最后一个字节表示位图右上角的像素。

//下面的代码可以完成第一个字节表示位图左上角的像素，而最后一个字节表示位图右下角的像素，即正常的显示状态，便于操作。

int m,n;

unsigned char k;

m = BMPPIC.BMPInfoHead.biWidth/8; //24

n = BMPPIC.BMPInfoHead.biHeight; //96, 24*96 = 2304 bytes

for(int i=0; i < n/2; i++)

{

for(int a=0; a < m; a++)

{

k = pbufout1[m*(n-i-1) + a];

pbufout1[m*(n-i-1) + a] = pbufout1[i*m + a];

pbufout1[i*m + a] = k;

}

}

存储算法

BMP文件通常是不压缩的，所以它们通常比同一幅图像的压缩图像文件格式要大很多。例如，一个800×600的24位几乎占据1.4MB空间。因此它们通常不适合在因特网或者其它低速或者有容量限制的媒介上进行传输。根据颜色深度的不同，图像上的一个像素可以用一个或者多个**字节**表示，它由n/8所确定（n是位深度，1字节包含8个**数据位**）。图片浏览器等基于字节的ASCII值计算像素的颜色，然后从调色板中读出相应的值。更为详细的信息请参阅下面关于**位图**文件的部分。n位2n种颜色的位图近似字节数可以用下面的公式计算：**BMP**文件大小约等于 54+4*2的n次方+（w*h*n)/8，其中高度和宽度都是像素数。需要注意的是上面公式中的54是位图文件的**文件头**，是彩色调色板的大小。另外需要注意的是这是一个近似值，对于n位的**位图图像**来说，尽管可能有最多2n中颜色，一个特定的图像可能并不会使用这些所有的颜色。由于彩色**调色板**仅仅定义了图像所用的颜色，所以实际的彩色调色板将小于。如果想知道这些值是如何得到的，请参考下面文件格式的部分。由于存储算法本身决定的因素，根据几个图像参数的不同计算出的大小与实际的文件大小将会有一些细小的差别。

数据类型

Oracle产品 EPC Bitmap 中数据类型 EPC bitmap的缩写。

Oracle提出了一种EPC bitmap的数据类型对基于RFID产品项级别跟踪应用产生的大量的数据进行有效处理，。

EPC bitmap数据类型定义一个EPC集合，集合共享EPC的一些特征（例如header,manager number,and object class）。支持这个数据类型的关键是使用RFID标识的物品项，在一个群组里可以基于共同属性（例如位置，截止日期，或制造商），在通常情况下可以被追踪。而EPC集合可以表示为一个EPC bitmap， bitmap可以被访问并可通过epc2bmp或bmp2epc进行数据类型的转换操作。

引入EPC bitmap（epc bmp）类型的好处：

(1) 可以简单的标识一个RFID EPC集合，而不会丢失任何信息；

(2) 对于同类的EPC操作可以简单的在bitmap上操作，简化了操作方式

br/

参考资料

1.

🔝

BMP 格式 🔗 . 百度百科[引用日期2012-11-03]

词条标签：非生物，互联网

猜你喜欢

- [bmp软件下载](#)
 - [国际快递](#)
- [bmp战车](#)
 - [mcake](#)
- [回农村创业](#)
 - [今日黄金行情](#)
- [幼儿英语](#)
 - [貂皮大衣](#)
- [药材种植](#)
 - [从零开始学炒股大全集](#)



人脸识别算法



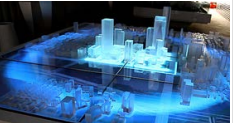
液晶拼接大屏幕



大数据分析培训



全息投影的手机



全息投影沙盘



无缝拼接液晶屏



三维激光扫描仪



新手上路

- [成长任务](#)
- [编辑入门](#)
- [编辑规则](#)
- [百科术语](#)

我有疑问

- [我要质疑](#)
- [在线客服](#)
- [参加讨论](#)
- [意见反馈](#)

投诉建议

- [举报不良信息](#)
- [未通过词条申诉](#)
- [投诉侵权信息](#)
- [封禁查询与解封](#)